

Approach

Architecture

Stateless FastAPI service (`/health`, `/chat`). Each `/chat` call: (1) build a retrieval query from the full message history, (2) BM25-search the 377-item catalog for the top ~25 candidates, (3) hand the LLM the conversation + only those candidates + a rules/few-shot system prompt, (4) parse its structured JSON decision, (5) **ground every recommended id against the local catalog before it ever reaches the response** – the LLM never emits a name or URL directly, only catalog ids, and any id it invents is silently dropped. This is the main defense against hallucinated URLs: correctness of the grounding step is enforced in code, not by asking the model nicely.

Retrieval

377 items is small. A full embeddings/vector-DB stack (Chroma/FAISS + sentence-transformers) adds cold-start weight and RAM on free hosting for no real recall benefit at this scale, so I used BM25 (`rank_bm25`) over name+description+category+job-level text, boosted 3x on name matches. Two gaps BM25 alone has: (a) short acronyms (“OPQ”, “GSA”, “DSI”) score poorly on term frequency, and (b) a user typing an exact product name should always surface that product. Both are handled by a small alias/substring force-include pass layered on top of the BM25 candidates (`app/retrieval.py`). Retrieval always returns a candidate *set*, never a final answer – final selection, coverage reasoning (“add a cognitive test as a default”), and refinement logic are the LLM’s job, grounded in that set.

Agent / prompt design

One LLM call per turn, returning strict JSON: `reply`, `in_scope`, `commit_shortlist`, `recommended_ids`, `end_of_conversation`. This maps onto the public schema in `app/agent.py`, with hard rules enforced in code regardless of what the model returns:

- `in_scope=false` (refusal) forces empty recommendations and `end_of_conversation=false`, **even if a shortlist was already committed earlier** – reading the 10 provided sample traces (esp. C7, C10) showed the reference behavior blanks recommendations on a refusal or a pure-negotiation turn and only re-shows the list once the agent explicitly reaffirms it. That distinction (state exists vs. state is being *repeated this turn*) turned out to be the single most important behavioral rule to get right, and it’s why `commit_shortlist` is a per-turn decision rather than “recommendations are empty only pre-first-commit.”
- Turn budget is tiny (8 total, so ~4 exchanges). The prompt explicitly tells the model to ask at most 1-2 clarifying questions before committing to a shortlist using sane defaults – without this instruction, early testing showed the model would happily keep asking clarifying questions past the turn cap, which zeroes out Recall@10 for that trace regardless of catalog quality.
- Compare questions (“what’s the difference between X and Y”) are answered only from the candidate list’s own descriptions, not general knowledge, and reuse the alias/substring matcher so an abbreviated product name still resolves to the right catalog row.

Two condensed few-shot examples (clarify-on-vague-input, refuse-mid- conversation-legal-question) are included directly in the system prompt rather than all 10 traces, trading a small amount of behavioral fidelity for prompt size and latency headroom against the 30s per-call limit.

What didn’t work / changed

- **test_type is derived, not scraped separately.** The provided catalog JSON only has a `keys` (category) field, not SHL’s internal single-letter “Test Type” badge. For most items these agree 1:1 (verified against the sample traces), but a few report-type products (e.g. “Global Skills Development Report”) show a narrower Test Type on shl.com than their full category list. I checked whether the live site could fill this gap and found it’s been restructured since the catalog was scraped (product URLs now 301-redirect elsewhere), so re-scraping would introduce fresh inconsistency rather than fix it. Since Recall@10 and the behavior probes score which assessments are recommended, not the accuracy of the `test_type` label, I mapped categories -> codes via SHL’s public Great-8 taxonomy and moved on rather than over-investing here.
- **Gemini 2.5 Flash silently returned empty content** under the default request until I added `reasoning_effort: "none"` – the model was spending the entire token budget on hidden reasoning tokens before emitting the JSON body, so a plain `max_tokens` bump alone didn’t fix it. Caught by the FastAPI smoke test returning an empty fallback reply, root-caused via the raw response’s `usage.completion_tokens: 0`.
- **Retry logic was originally uniform** (retry any exception with a “please output valid JSON” follow-up). Split it: JSON-shape errors get one repair round-trip; transport/rate-limit errors (429/timeout) fail straight to the safe fallback, since a same-request retry can’t out-wait a 429 that says “retry in 50s” within a 30s call budget.

- **Prompt size vs. free-tier token limits.** The original 40-candidate format (~5,000 input tokens/call) was fine on paper but repeatedly tripped Groq’s free-tier 12,000-tokens/minute cap during eval runs. Cut candidates to 25 and compressed each row (dropped the `always-true remote` field entirely, replaced repeated category names with a one-line legend + single-letter codes, shortened descriptions to 100 chars) – roughly halved prompt size (~2,800 tokens/call) with no measured drop in which items got retrieved, since the alias/substring force-include still guarantees exact-name and acronym matches regardless of BM25 cutoff.

Evaluation

Measured against all four axes the task calls out, each with a runnable script/test suite rather than a one-off judgment call:

- **Retrieval quality** – `tests/test_retrieval.py` asserts BM25 surfaces relevant items for a query and the alias layer resolves acronyms/exact names that term-frequency search alone misses (e.g. “GSA”, “OPQ”).
- **Recommendation relevance** – `eval/run_eval.py` replays the 10 provided sample conversations’ user turns (verbatim) against the live agent and reports Recall@10 per trace by URL match. **Result: mean Recall@10 = 0.56** across the 7 traces I could score before hitting Groq’s free-tier daily token cap (100k/day) mid-run (see limitation below); no trace scored 0 on a real (non-rate-limited) run. Reference shortlists are often more specific than a reasonable first-pass answer (e.g. trace C1 expects the exact OPQ32r + two OPQ report add-ons; the agent picked three different, individually-defensible leadership-benchmark assessments) – partial credit across nearly every trace, rather than exact matches, is the realistic target for a system with no query-specific tuning.
- **Groundedness** – enforced structurally, not just measured: `_ground_recommendations()` in `app/agent.py` filters any id the LLM returns that isn’t in that turn’s candidate list, so a hallucinated recommendation is *impossible* to return, not just unlikely. `test_hallucinated_ids_only_downgrades_to_no_commit` in `tests/test_agent.py` verifies this with a stub LLM that deliberately hallucinates, and the `groundedness_urls_are_real_catalog_urls` probe in `eval/behavior_probes.py` checks it live end-to-end (passed).
- **Overall behavior/response accuracy** – `eval/behavior_probes.py` runs 8 targeted single/multi-turn probes mirroring the assignment’s own examples (refuses off-topic, refuses legal questions, refuses prompt injection, no recommendation on a vague turn-1 query, commits a shortlist for a detailed request, refines on a new constraint, always-valid schema even on empty/garbage input). **Result: 7/8 passed** on a live run; the one “failure” (`refine_updates_shortlist_on_new_constraint`) was the agent declining to bolt on a redundant personality test because the existing recommended item already covers Personality & Competencies – arguably correct judgment, and a sign the probe’s assertion is stricter than it should be, not that the agent is wrong.

Both eval scripts detect and skip (not silently pass) turns where the LLM call itself failed transport-wise (rate limit/timeout), so a quota exhaustion mid-run is reported as “N inconclusive” rather than misread as a real 0%. `tests/` (15 tests) covers everything deterministic – grounding, refusal overrides, malformed-JSON repair, schema shape – against a stub LLM client, so those don’t depend on any provider or quota at all.

Known limitation at submission time: a full clean Recall@10 run across all 10 traces was blocked by Groq’s free-tier daily token cap (100k tokens/day; this eval alone uses ~3k tokens/turn x ~30 turns), which was hit partway through – 7/10 traces scored, 3 skipped as inconclusive rather than scored as failures. The earlier draft of this document used a Gemini key whose free tier turned out to be even tighter (~20 requests/day) and which also silently returned empty completions until `reasoning_effort: "none"` was set (see above) – switching to Groq fixed both the token-truncation bug and gave enough headroom to get the real numbers above. For grading traffic beyond what either free tier allows, a paid tier on either provider removes the constraint entirely; the code doesn’t change.

AI tool usage

Used Claude Code as a pair-programmer for the full implementation (retrieval, agent orchestration, prompt design, tests, eval harness) and for reading the provided PDF/JSON/sample-conversation assets to reverse-engineer the exact expected behaviors (e.g. the refusal-blanks-recommendations rule above, which isn’t stated explicitly in the assignment PDF and was only visible by reading trace C7 closely). All design tradeoffs above were deliberate calls made during that process, not left as defaults.